

Excel Importer — изменения в коде

Сравнение кода до и после доработки · 29.09.2026

ОБЛАСТЬ	БЫЛО	СТАЛО
Загрузка файла	Весь файл читался в <code>UploadController</code> прямо в HTTP-запросе: <code>IOFactory::load()</code> + <code>toArray()</code> . Он целиком загружался в память, и эти данные передавались в джобы. Маршрут стоял в группе <code>web</code> , поэтому реальный запрос получал 419 (CSRF). Валидация — только <code>mimes:xlsx</code> .	<code>ImportController@store</code> : проверка файла (<code>extensions</code> , <code>mimes</code> по содержимому, <code>max</code>) → сохранение → запись <code>Import</code> → ответ 202. Вся работа уходит в очередь. Маршрут в <code>api</code> , CSRF не мешает.
Чтение и разбиение на чанки	Весь файл в массиве, <code>array_chunk</code> , обычный <code>dispatch()</code> без батча. Момент окончания импорта не определялся.	<code>SpreadsheetReader</code> читает файл потоково (OpenSpout). <code>ReadImportFile</code> добавляет чанки в <code>Bus::batch</code> через <code>batch()->add()</code> , в <code>finally()</code> импорт завершается.
Обработка чанка	- Номер строки вычислялся из текущего значения счётчика в Redis — при параллельной обработке неверный. - На каждую строку отдельные <code>exists()</code> + <code>create()</code> — 2000 запросов на чанк. - Между проверкой и вставкой была гонка: падение на уникальном индексе → ретрай → двойной подсчёт. <code>is_numeric</code> пропускал <code>1.5</code> и <code>1e5</code>. - Событие <code>RowCreated</code> на каждую строку, с отдельной джобой в очереди.	- У каждой строки свой реальный номер из Excel. - Отдельный <code>RowValidator: id</code> проверяется как строка (весь диапазон <code>unsigned bigint</code>), дата — через <code>checkdate</code>. - Один <code>insertOrIgnore (ON CONFLICT DO NOTHING)</code> на чанк, ключи отсортированы, чтобы не было дедлоков. - Дубликат определяется по тому, какой импорт и какая строка файла первыми записали этот <code>id</code>. - Одно событие <code>RowsCreated</code> на чанк, отправляется сразу, без очереди.
Прогресс	Ключ <code>import_progress:md5(path)</code> только с числом обработанных строк. Нет общего количества и статуса, при ретрае прогресс удваивается.	Класс <code>ImportProgress</code> : в Redis ключи <code>processed</code> , <code>failed</code> , <code>chunks</code> , <code>errors</code> . Lua-скрипт засчитывает каждый чанк ровно один раз. В таблице <code>imports</code> — статус, общее число строк и итоговые счётчики.

ОБЛАСТЬ	БЫЛО	СТАЛО
Отчёт	Один общий <code>result.txt</code> на все импорты. Никогда не очищался, ошибки дописывались туда из джоб в случайном порядке.	У каждого импорта свой <code>imports/{uuid}/result.txt</code> . Собирается один раз в конце, отсортирован по номеру строки, скачивается через API.
Выдача данных	<code>Row::all()->groupBy()</code> загружал всю таблицу в память и падал на <code>\$row->date->format()</code> — у модели не было <code>cast</code> .	<code>RowController</code> : выборка с пагинацией, <code>cast date:d.m.Y</code> в модели. Ответ — двумерный массив «дата → строки».
База данных	MySQL. В <code>rows</code> автоинкрементный <code>id</code> + <code>external_id</code> типа <code>unsignedBigInteger</code> (в PostgreSQL это знаковый <code>bigint</code> — большие <code>id</code> не помещаются).	PostgreSQL. <code>rows.id</code> — <code>numeric(20,0)</code> и первичный ключ, добавлены <code>import_id</code> , <code>row_number</code> и индекс по <code>date</code> . Новая таблица <code>imports</code> с <code>uuid</code> .
Авторизация	Логин и пароль <code>admin/secret</code> захардкожены, сравнение через <code>!==</code> .	Данные из конфига, сравнение через <code>hash_equals</code> . Добавлены лимит загрузок по IP и доверие к прокси.
Фронтенд	<code>bootstrap.js</code> вызывал <code>require()</code> внутри Vite — сборка падала. Echo подключался к несуществующему <code>laravel-websockets</code> .	Vue 3, две страницы. Echo подключается к Reverb, а если WebSocket недоступен — опрос API.
Джобы	Нет трейтов <code>Queueable</code> и <code>Batchable</code> , нет <code>tries</code> , <code>backoff</code> , <code>timeout</code> , <code>failed()</code> .	Всё это есть. Воркер периодически перезапускается (<code>--max-jobs</code> , <code>--max-time</code> , <code>--memory</code>).
Качество	2 теста, <code>strict_types</code> нигде нет, типов почти нет.	89 тестов на настоящих PostgreSQL и Redis, <code>declare(strict_types=1)</code> во всех файлах, типы везде. Pint и Larastan уровня 8 проходят без ошибок.
Зависимости	PhpSpreadsheet 4 (Composer блокирует его из-за уязвимостей), predis, Sanctum и Sail (не использовались).	OpenSpout, phpredis, Reverb, Larastan.